

GPU computing in discrete optimization. Part II: Survey focused on routing problems

Christian Schulz · Geir Hasle · André R. Brodtkorb ·
Trond R. Hagen

Received: 29 November 2012 / Accepted: 28 March 2013

© Springer-Verlag Berlin Heidelberg and EURO - The Association of European Operational Research Societies 2013

Abstract In many cases there is still a large gap between the performance of current optimization technology and the requirements of real-world applications. As in the past, performance will improve through a combination of more powerful solution methods and a general performance increase of computers. These factors are not independent. Due to physical limits, hardware development no longer results in higher speed for sequential algorithms, but rather in increased parallelism. Modern commodity PCs include a multi-core CPU and at least one GPU, providing a low-cost, easily accessible heterogeneous environment for high-performance computing. New solution methods that combine task parallelization and stream processing are needed to fully exploit modern computer architectures and profit from future hardware developments. This paper is the second in a series of two. Part I gives a tutorial style introduction to modern PC architectures and GPU programming. Part II gives a broad survey of the literature on parallel computing in discrete optimization targeted at modern PCs, with special focus on routing problems. We assume that the reader is familiar with GPU programming, and refer the interested reader to Part I. We conclude with lessons learnt, directions for future research, and prospects.

Keywords Discrete optimization · Parallel computing · Heterogeneous computing · GPU · Survey · Introduction · Tutorial · Transportation · Travelling salesman problem · Vehicle routing problem

C. Schulz · G. Hasle (✉) · A. R. Brodtkorb · T. R. Hagen
Department of Applied Mathematics, SINTEF ICT, Blindern, P.O. Box 124, 0314 Oslo, Norway
e-mail: geir.hasle@sintef.no

Introduction

In Part I of this paper (Brodtkorb et al. 2013), we give a brief introduction to parallel computing in general and describe modern computer architectures with multi-core processors for task parallelism and accelerators for data parallelism (stream processing). A simple prototype of a GPU-based local search procedure is presented to illustrate the execution model of GPUs. Strategies and guidelines for software development and performance optimization are given. On this background, we here, in Part II, give a survey of the existing literature on parallel computing in discrete optimization targeted at modern PC platforms. With few exceptions, the work reported focuses on GPU parallelization.

The section “[Literature survey with focus on routing problems](#)” contains the bulk of Part II. It starts with an overall description of our literature search before we refer to “[Early works on non-GPU related accelerators](#)”. The rest of the section is structured according to type of optimization method. As a reflection of the number of publications, the first and most comprehensive part concerns metaheuristics. We give accounts of the literature on “[Swarm intelligence metaheuristics \(routing\)](#)”, “[Population based metaheuristics \(routing\)](#)” and “[Local search and trajectory-based metaheuristics \(routing\)](#)”, respectively. For all optimization methods, we briefly describe the method in question, present a survey of papers, often also in tabular form, and synthesize the insights gained. The section ends with a discussion of “[GPU computing for shortest path problems](#)”. In “[Literature on non-routing problems](#)” we give an overview over GPU implementations of metaheuristics applied to problems that are not related to routing, using the same structure as in the previous section. In addition we discuss “[Hybrid metaheuristics](#)”. As Linear Programming and Branch & Bound are important bases for methods in discrete optimization, we give a brief account of “[GPU implementations of Linear Programming and Branch & Bound](#)”. We conclude Part II with “[Lessons for future research](#)” followed by a “[Summary and conclusion](#)”.

Literature survey with focus on routing problems

Parallel methods to alleviate the computational hardness of discrete optimization problems (DOPs) are certainly older than the modern PC architecture. Parallelized heuristics, metaheuristics, and exact methods for DOP have been investigated since the 1980s and there is a voluminous literature, see for instance Talbi (2006) and Alba (2005) for general surveys, and Crainic (2008) for a survey focused on the VRP. Most of the work is based on task parallelism, but the idea of using massive data parallelism to speed up genetic algorithms dates back to the early 1990s, see for instance Spiessens and Manderick (1991).

It should be clear that population-based metaheuristics and methods from swarm intelligence such as ant colony optimization lend themselves to different types of parallelization at several levels of granularity. Both task and data parallelization are possible, and within both types there are many alternative parallelization schemes. Also, the neighborhood exploration in local search that is the basis and the

bottleneck of many trajectory-based metaheuristics is inherently parallel. At a fine-grained level, the evaluation of objective components and constraints for a given neighbor may be executed in parallel. A more coarse-grained parallelization results from neighborhood splitting. What may be regarded as the simplest metaheuristic—multi-start local search—is embarrassingly parallel. Again, both data and task parallelization may be envisaged, and there are many non-trivial design decisions to make including the parallelization scheme.

Branch & Bound and Branch & Cut are basic tree search algorithms in exact methods for DOP. At least conceptually, they are easy to parallelize, but load balancing and scaling are difficult issues. We refer to Crainic et al. (2006) and Ralphs (2006) for in-depth treatments. Commercial MILP solvers typically have task parallel versions that are well suited for multi-core processors. As far as we know there exists no commercial MILP solver that exploits stream processing accelerators. More sophisticated exact MILP methods such as Branch & Cut & Price are harder to parallelize (Ralphs et al. 2003).

In a literature search (2012), we found some 100 publications on GPU computing in discrete optimization. They span the decade 2002–2012. With only a few exceptions they discuss GPU implementation of well-known metaheuristics, or problem-specific special algorithms. Very few address the combined utilization of the CPU and the GPU. Below, we give some overall characterizations of the publications found, before we structure and discuss the literature in some detail.

As for applications and DOPs studied, 28 papers describe research on routing problems, of which 9 focus on the shortest path problem (SPP), 16 discuss the TSP, and only 3 study the VRP. As GPU computing for the SPP is peripheral to the goals of this paper, we only give a brief survey of the literature in “[GPU computing for shortest path problems](#)”. Also relevant to transportation there is a paper on route selection for car navigation (Bura et al. 2011), and one on route planning in aerial surveillance (Sanci and Isler 2011). Bleiweiss (2008) describes an efficient GPU implementation of parallel global pathfinding using the CUDA programming environment. The application is real-time games where a major challenge is autonomous navigation and planning of thousands of agents in a scene with both static and dynamic moving obstacles. Rostrup et al. (2011) describe a GPU implementation of Kruskal’s algorithm for the minimum spanning tree problem.

Amongst others, the following DOPs and applications are studied:

- Allocation of tasks to heterogeneous processing units
- Task matching
- Flowshop scheduling
- Option pricing
- FPGA placement
- VLSI circuit optimization
- Protein sequence alignment in bioinformatics
- Sorting
- Learning
- Data mining
- Permutation perceptron problem

- Knapsack problem
- Quadratic assignment problem
- 3-SAT and Max-SAT
- Graph coloring

Not surprisingly, the bulk of the literature with some 80 papers discusses implementations of swarm intelligence methods and population-based metaheuristics. Of the 41 swarm intelligence papers found, there are 23 on ant colony optimization (ACO) and 18 on particle swarm optimization (PSO). Most of the PSO publications focus on continuous optimization. The identified literature on population-based metaheuristics (evolutionary algorithms, genetic algorithms, genetic programming, memetic algorithms, and differential evolution) also consists of 41 publications. The remaining publications cover (number of publications) the following:

- Metaheuristics in general (1)
- Immune systems (2)
- Local search (8)
- Simulated annealing (3)
- Tabu search (3)
- Special purpose algorithms (2)
- Linear programming (4)

The most commonly used basis for justifying a GPU implementation is speed comparison with a CPU implementation. This is useful as a first indication, but it is not sufficient by itself. Important aspects such as the utilization of the GPU hardware are typically not taken into consideration. Moreover, the CPU code used for comparison is normally unspecified and thus unknown to the reader. We refer to “[Lessons for future research](#)” for a detailed discussion on speedup comparison. Often, an algorithm can be organized in different ways, which in turn can have a variety of GPU implementations, each using different GPU specifics such as shared memory. Only a few papers discuss and compare different algorithmic approaches on the GPU. A thorough investigation of hardware utilization, e.g., through profiling of the implemented kernels, is missing in nearly all of the papers. For these, we will simply quote the reported speedups. If a paper provides more information on the CPU implementation used, different approaches, or profiling, we will mention this explicitly.

Early works on non-GPU related accelerators

Early papers utilize hardware such as field-programmable gate arrays (FPGAs). Guntsch et al. (2002) is the earliest paper in our survey. It appeared in 2002 and proposes a design for an ACO variant, called population-based ACO (P-ACO), that allows efficient FPGA implementation. In Scheuermann et al. (2004), an overlapping set of authors report from the actual implementation of the P-ACO design. They conduct experiments on random instances of the single-machine total tardiness problem (SMTTP) with number of jobs ranging from 40 to 320 and report moderate

speedups between 1.6 and 10 relative to a software implementation. In Scheuermann et al. (2007), they continue their work on ACO for FGPAs and propose a new ACO variant, called counter-based ACO. The algorithm is designed such that it can easily be mapped to FPGAs. In simulations they apply this new method to the TSP.

Swarm intelligence metaheuristics (routing)

The emergent collective behavior in nature, in particular the behavior of ants, birds, and fish is the inspiration behind swarm intelligence metaheuristics. For an introduction to swarm intelligence, see for instance Kennedy et al. (2001). Swarm intelligence metaheuristics are based on communication between many, but relatively simple, agents. Hence, parallel implementation is a natural idea that has been investigated since the birth of these methods. However, there are non-trivial design issues regarding parallelization granularity and scheme. A major challenge is to avoid communication bottlenecks.

The methods of this category that we have found in the literature of GPU computing in discrete optimization are ACO, PSO, and flocking birds (FB). ACO is the most widely studied swarm intelligence metaheuristic (23 publications), followed by PSO (18) and FB (3). ACO is also the only swarm intelligence method applied to routing problems in our survey, which is why we will discuss it here. For an overview of GPU implementations of the other swarm intelligence methods, we refer to “[Swarm intelligence metaheuristics \(non-ACO, non-routing\)](#)”.

In ACO, there is a collection of ants where each ant builds a solution according to a combination of cost, randomness and a global memory, the so-called pheromone matrix. Applied to the TSP this means that each ant constructs its own solution. Afterwards, the pheromone matrix is updated by one or more ants placing pheromone on the edges of its tour according to solution quality. To avoid stagnation and infinite growth, there is a pheromone evaporation step added before the update, where all existing pheromone levels are reduced by some factor. There exist variants of ACO in addition to the basic ant system (AS). In the max–min ant system (MMAS), only the ant with the best solution is allowed to deposit pheromone and the pheromone levels for each edge are limited to a given range. Proposed by Stützle, the MMAS has proven to be one of the most efficient ACO metaheuristics. The most studied problem with ACO is the TSP. There are also several ACO papers on the SPP and variants of the VRP.

Parallel versions of ACO have been studied extensively in the literature, and several concepts have been developed. The two predominant, basic parallelization schemes are parallel ants, where one process/thread is allocated to each ant, and the multiple colonies approach. Pedemonte et al. (2011) introduce a new taxonomy for classifying parallel ACO algorithms and also present a systematic survey of the current state-of-the-art on parallel ACO implementations. As part of the new taxonomy they describe the master-slave category, where a master process manages global information and slave processes perform subordinate tasks. This concept can again be split into coarse-grained and fine-grained. In the former, the slaves compute whole solutions, as done in parallel ants. In the latter, the slaves only perform parts of the computation for one solution. Pedemonte et al. consider a wide

Table 1 ACO implementations on the GPU related to routing

References	Problem	Algorithm	GPU (s)	Tour construction		Ph. update		Max. speedup	CPU code
				GP	CUDA: one-ant-per	GP	CUDA		
Catala et al. (2007)	OP	ACO	GeForce 6600 GT	x					Mocholí et al. (2005)
Bai et al. (2009)	TSP	Multi colony MMAS	GeForce 8800 GTX		x		x	2.3	?
Li et al. (2009a)	TSP	MMAS	GeForce 8600 GT			x	x	11	?
Wang et al. (2009)	TSP	MMAS	Quadro Fx 4500	x				1.1	?
You (2009)	TSP	ACO	Tesla C1060		x			21	?
Cecilia et al. (2011)	TSP	ACO	Tesla C2050		x	x	x	29	Dorigo and Stützle (2004)
Delévacq et al. (2013)	TSP	MMAS & multi-colony	2 × Tesla C2050		x	x	-/x	23.6	?
Diego et al. (2012)	VRP	ACO	GeForce 460 GTX		x		x	12	?
Uchida et al. (2012)	TSP	AS	GeForce 580 GTX			x	x	43.5	Own

OP orienteering problem, Ph. pheromone, GP graphics pipeline, -/x, in some settings; ?, unknown CPU code

variety of parallel computing platforms. However, out of the 69 publications surveyed, only 13 discuss multi-core CPU (9) and GPU platforms (4). Table 1 presents an overview of the routing-related GPU papers implementing ACO that we found in the literature, showing which steps of ACO are performed on the GPU in what fashion and by which paper.

ACO exhibits apparent parallelism in the tour construction phase, as each ant generates its tour independently. The inherent parallelism has led to early implementations of this phase on the GPU using the graphics pipeline. In Catala et al. (2007) and Wang et al. (2009), fragment shaders are used to compute the next city selection. In both papers, the necessary data is stored in textures and computational results are made available by render-to-texture, enabling later iterations to use earlier results. Wang et al. (2009) assign to each ant-city combination a unique (x, y) pixel coordinate and only generate one fragment per pixel. This leads to a conceptually simple setup that needs multiple passes to compute the result. Catala et al. (2007) relate one pixel to an ant at a certain iteration and generate one fragment per city related to this pixel. The authors utilize depth testing to select the next city and also provide an alternative implementation of tour construction using a vertex shader.

With the arrival of CUDA and OpenCL, programming the GPU became easier and consequently more papers studied ACO implementations on the GPU. In CUDA and OpenCL there is the basic concept of having a thread/workitem as basic computational element. Several of them are grouped together into blocks/workgroups. For convenience we will use the CUDA language of threads and blocks. From the parallel master-slave idea, one can derive two general approaches for the tour construction on the GPU. Either a thread is assigned to computing the full tour of one ant, or one thread computes only part of the tour and a whole thread block is assigned per ant. Thus we have the one-ant-per-thread and the one-ant-per-block schemes. Many papers implement either the former (Bai et al. 2009; You 2009; Diego et al. 2012) or the latter (Li et al. 2009a; Uchida et al. 2012). Only a few publications (Cecilia et al. 2011; Delévacq et al. 2013) compare the two. Cecilia et al. argue that the one-thread-per-ant approach is a kind of task parallelization and that the number of ants for the studied problem size is not enough to fully exploit the GPU hardware. Moreover, they argue that there is divergence within a warp and that each ant has an unpredictable memory access pattern. This motivated them to study the one-block-per-ant approach as well.

Most papers provide a single implementation of their selected approach, often reporting how they use certain GPU specifics such as shared and constant memory. In contrast, the papers by Cecilia et al. (2011), Delévacq et al. (2013), and Uchida et al. (2012) study different implementations of at least one of the approaches. For the one-ant-per-thread scheme, Cecilia et al. (2011) examine the effects of separating the computation of the probability for each city from the tour construction. They also introduce a list of nearest neighbors that have to be visited first to reduce the amount of random numbers. The effects of shared memory and texture memory usage are studied. Delévacq et al. also examine the effects of using or not using shared memory. Moreover, they study the addition of a local search step to improve each ant's solution. Uchida et al. (2012) examine different approaches of

city selection in the tour construction step to reduce the amount of probability summations.

As the pheromone update step is often less time consuming than the tour construction step, not all papers put it on the GPU. Most of the ones that do investigate only a single pheromone update approach. In contrast, Cecilia et al. (2011) propose different pheromone update schemes and investigate different implementations of those schemes.

An additional parallelization concept developed already in the pre-GPU literature is multi-colony ACO. Here, several colonies independently explore the search space using their own pheromone matrices. The colonies can cooperate by periodically exchanging information (Pedemonte et al. 2011). On a single GPU this approach can be realized by assigning one colony per block, as done by Bai et al. (2009) and by Delévacq et al. (2013). If several GPUs are available, one can of course use one GPU per colony as studied by Delévacq et al. (2013).

Both Catala et al. (2007) and Cecilia et al. (2011) provide information about the CPU implementation used for computing the achieved speedups, see Table 1. Catala et al. compare their implementations against the GRID-ACO-OP algorithm (Mocholí et al. 2005) running on a grid of up to 32 Pentium IV.

From the above description, we observe that for the ACO, the task most commonly executed on the GPU is tour construction. The papers of Cecilia et al. (2011) and Delévacq et al. (2013) indicate that the one-ant-per-block scheme seems to be superior to the one-ant-per-thread scheme.

Population-based metaheuristics (routing)

By population-based metaheuristics we understand methods that maintain and evolve a population of solutions, in contrast with trajectory (or single solution)-based metaheuristics that are typically based on local search. In this subsection we will focus on evolutionary algorithms. For a discussion of swarm intelligence methods on the GPU we refer to the “[Swarm intelligence Metaheuristics \(routing\)](#)” above.

In evolutionary algorithms, a population of solutions evolves over time, yielding a sequence of generations. A new population is created from the old one using a process of reproduction and selection, where the former is often done by crossover and/or mutation and the latter decides which individuals form the next generation. A crossover operator combines the features of two parent solutions to create children. Mutation operators simply change (mutate) one solution. The idea is that, analogous to natural evolution, the quality of the solutions in the population will increase over time. Evolutionary algorithms provide clear parallelism. The computation of offspring can be performed with at most two individuals (the parents). Moreover, the crossover operators might be parallelizable. Either way, enough individuals are needed to fully saturate the GPU, but at the same time all of them have to make a contribution to increasing the solution quality (see, e.g. Fujimoto and Tsutsui (2011)).

In our literature search, we found publications on evolutionary algorithms (EA) and genetic algorithms (GA) (25), genetic programming (12), and differential

Table 2 Overview of EA GPU implementations on the GPU for routing

References	Problem	Algorithm	Operators	Selection		GPU (s)	Max. speedup	CPU code
				Immune	Next population			
Li et al. (2009b)	TSP	IEA	PMX, mutation	Better	Tournament	GeForce 9600 GT	11.5	?
Chen et al. (2011)	TSP	GA	crossover, 2-opt mutation		Best	Tesla C2050	1.7	?
Fujimoto and Tsutsui (2011)	TSP	GA	OX, 2-opt local search gene string move		Best	GeForce GTX 285	24.2	?
Zhao et al. (2011)	TSP	IEA	Multi bit exchange	Best position	Tournament	GeForce GTS 250	7.5	?

IEA immune EA, *GA* genetic algorithm, *PMX* partially mapped crossover, *OX* order crossover, *Better* best of vaccinated and not vaccinated tour, *Best position* vaccine creates set of solutions, best is chosen, *Best* best of parent and child, *?*, unknown CPU code

Table 3 Studied implementation approaches with respect to whether one individual is assigned to one thread or block

References	Crossover		Mutation		Vaccination		Tour evaluation	
	T	B	T	B	T	B	T	B
Li et al. (2009b)								x
Chen et al. (2011)	x		x					
Fujimoto and Tsutsui (2011)		x		x				x
Zhao et al. (2011)			x			U		

T thread, *B* block, *U* uncertain

evolution (3) within this category. For combinations of EA/GA with LS, and memetic algorithms, see “[Hybrid metaheuristics](#)” below.

Although the literature is rich on GPU implementations of population-based metaheuristics, only a few publications discuss routing problems. The ones we found are all presented in Table 2. They use either a genetic algorithm or an immune evolutionary algorithm which combines concepts from immune systems¹ with evolutionary algorithms. All the papers we have found in this category use CUDA.

In some of the GPU implementations, the crossover operator is completely removed to avoid binary operations and yield totally independent individuals. In the routing-related GPU literature the apparent parallelism has led to the two parallelization schemes of assigning one individual to one thread (coarse grained parallelism) (Chen et al. 2011) and one individual to one block (fine grained parallelism) (Li et al. 2009b; Fujimoto and Tsutsui 2011), see also Table 3. In some papers, different parallelization schemes are used for different operators. We have seen no paper that directly compares both schemes for the same operation.

The scheme chosen obviously influences the efficiency and quality of the GPU implementation. On the one hand a minimum number of individuals is needed to fully saturate all of the computational units of the GPU, especially with the one-individual-per-thread scheme. On the other hand, from an optimization point of view, it might not increase the quality of the algorithm to have a huge population size (Fujimoto and Tsutsui 2011). Analogously, the one-individual-per-block scheme only makes sense if the underlying operation can be distributed over the threads of a block.

Most of the papers describe their approach with details on the implementation. Zhao et al. (2011) compare their work in addition with the results of four other papers (Acan 2002; Li et al. 2008, 2009a, b). They report that their own implementation has the shortest GPU running time, but interestingly the speedup compared with unknown CPU implementations is highest for Li et al. (2009b).

Local search and trajectory-based metaheuristics (routing)

Local search (LS, neighborhood search), see for instance Aarts and Lenstra (2003), is a basic algorithm in discrete optimization and trajectory-based metaheuristics. It

¹ Artificial immune systems (AIS) is a sub-field of Biologically-inspired computing. AIS is inspired by the principles and processes of the vertebrate immune system.

is the computational bottleneck of single solution-based metaheuristics such as tabu search, guided local search, variable neighborhood search, iterated local search, and large neighborhood search. Given a current solution, the idea in LS is to generate a set of solutions—the neighborhood—by applying an operator that modifies the current solution. The best (or, alternatively, an improving) solution is selected, and the procedure continues until there is no improving neighbor, i.e., the current solution is a local optimum. An LS example is described in Part I (Brodtkorb et al. 2013).

The evaluation of constraints and objective components for each solution in the neighborhood is an embarrassingly parallel task, see for instance Melab et al. (2006) and Brodtkorb et al. (2013) for an illustrating example. Given a large enough neighborhood, an almost linear speedup of neighborhood exploration in LS is attainable. The massive parallelism in modern accelerators such as the GPU seems well suited for neighborhood exploration. This has naturally led to several research papers implementing local search variations on the GPU, reporting speedups of one order of magnitude when compared with a CPU implementation of the same algorithm. Profiling and fine-tuning the GPU implementation may ensure good utilization of the GPU. Schulz (2013) reports a speedup of up to one order of magnitude compared with a naive GPU implementation. To fully saturate the GPU, the neighborhood size is critical; it must be large enough (Schulz 2013). The effort of evaluating all neighbors can be exploited more efficiently than by just applying one move. In Burke and Riise (2012) a set of improving and independent moves is determined heuristically and applied simultaneously, reducing the number of neighborhood evaluations needed.

We would have liked to present clear guidelines for implementing LS on the GPU based on the observed literature. Due to the richness of applications, problems, and variations of LS, this is not possible. Instead, we shall discuss approaches taken in papers that study routing problems.

Although the term originates from genetic algorithms, we will use the term *fitness structure* for the collection of delta values (see Section 5 in Brodtkorb et al. 2013) and feasibility information for all neighbors of the current solution. Table 4 provides an overview of the routing-related GPU papers using some kind of local search. The earliest by Janiak et al. (2008) utilizes the graphics pipeline for tabu search by providing a fragment shader that evaluates the whole neighborhood in a one fragment per move fashion. The remaining steps of the search were performed on the CPU.

With the availability of CUDA, the number of papers studying LS and LS-based metaheuristics on the GPU increased. The technical report by Luong et al. (2009) discusses a CUDA-based GPU implementation of LS. To the authors' best knowledge, this is the first report of a GPU implementation of pure LS. Further research is discussed in two follow-up papers (Luong et al. 2011a, b). The authors apply LS to different instances of well-known DOPs such as the quadratic assignment problem and the TSP. We will concentrate on their results for routing related problems, i.e., the TSP.

Table 4 Overview of LS-based GPU literature on routing

References	Problem	Algorithm	Neighborhood	Approach	GPU (s)	Max. speedup	CPU code
Janiak et al. (2008)	TSP	TS	2-exchange (swap)	Graphics pipeline: move evaluation by fragment shader	GeForce 8600 GT	1.12	C#
Luong et al. (2011b)	TSP	LS	2-exchange (swap)	CUDA	a.o. Tesla M2050	19.9	?
O'Neil et al. (2011)	TSP	MS-LS	2-opt	CUDA: multiple-Is-per-thread, load balancing	Tesla C2050	61.9	Single core
Burke and Riise (2012)	TSP	ILS	VND: 2-opt + relocate	CUDA: one-move-per-thread, applies several independent moves at once	GeForce GTX 280	70×7.5	?
Coelho et al. (2012)	SVRPDSP	VNS	Swap + relocate	CUDA: one-move-per-thread	Geforce GTX 560 Ti	17	own
Rocki and Suda (2012)	TSP	(LS)	2-opt, 3-opt	CUDA: several-moves-per-thread	a.o. Geforce GTX 680	27	32 cores
Schulz (2013)	DCVRP	LS	2-opt, 3-opt	CUDA: one-move-per-thread, asynchronous execution, very large nbhs	GeForce GTX 480		

TS tabu search, LS local search, MS-LS multi start local search, ILS iterated local search, VND variable neighborhood descent, VNS variable neighborhood search, SVRPDSP single vehicle routing problem with deliveries and selective pickups, a.o. amongst others (only maximum speedup is mentioned), (LS) Rocki and Suda consider only the neighborhood evaluation part

Local search on the GPU

Thanks to the flexibility and ease of programming of CUDA, more steps of the LS process can be executed on the GPU. Table 5 provides an overview of what steps are done on the GPU in which routing-related publication. Table 6 shows the CPU-GPU copy operations involved. Broadly speaking, LS consists of neighborhood generation, evaluation, neighbor/move selection, and solution update. The first task can be done in several ways. A simple solution is to generate the neighborhood on the CPU and copy it to the GPU on each iteration. Alternatively, one may create the neighborhoods directly on the GPU. The former approach, taken by Luong et al. (2011b), involves copying a lot of information from the CPU to the GPU on each iteration.

The neighborhood is normally represented as a set of moves, i.e., specific changes to the current solution. If one thread on the GPU is responsible for the evaluation of one or several moves, a mapping between moves and threads can be provided. This mapping can either be an explicit formula (Luong et al. 2011b; Burke and Riise 2012; Coelho et al. 2012; Rocki and Suda 2012; Schulz 2013) or an algorithm (Luong et al. 2011b). Alternatively, it can be a pre-generated explicit

Table 5 Tasks performed on the GPU during one iteration

References	Nbh gen.	Nbh eval.	Neighbor sel.	Sol. update
Janiak et al. (2008)	i	x		
Luong et al. (2011b)	−/i	x	−/x	
O’Neil et al. (2011)	i	x	x	x
Burke and Riise (2012)	i	x		
Coelho et al. (2012)	i	x		x
Schulz (2013)	i	x	x	x

Nbh neighborhood, *gen.* generation, *eval.* evaluation, *sel.* selection, *Sol* solution, *i* neighborhood generation is done implicitly (use of some nbh description), *−/x* done in some settings, *−/i* done in some settings

Table 6 Data copied from and to GPU

References	Once		In each iteration			
	Prob. desc.	Nbh. desc.	Sol.	Nbh.	FS	Sel. move
Janiak et al. (2008)	↑	↑	↑		↓	
Luong et al. (2011b)	↑		↑	−/↑	−/↓	−/↓
O’Neil et al. (2011)	↑					
Burke and Riise (2012)	↑		↑		s↓	
Coelho et al. (2012)	↑				↓	↑
Schulz (2013)	↑	↑				↓

Prob. problem, *Nbh.* neighborhood, *desc.* description, *Sol.* solution, *FS* fitness structure, *Sel.* selected, ↑, upload to GPU from CPU; ↓, download from GPU to CPU; s↓, subset of fitness structure is downloaded from GPU; −/↑ or −/↓, copied in some settings

mapping that lies in the GPU memory as investigated by Janiak et al. (2008) and Schulz (2013). The advantage of the mapping approach is that there is no need for copying any information to the GPU on each iteration. The pre-generated mapping only needs to be copied to the GPU once before the LS process starts.

The neighborhood evaluation is the most computationally intensive task in LS-based algorithms. Hence, all papers perform this task on the GPU. In contrast, selecting the best move is not always done on the GPU. A clear consequence of CPU-based move selection is the necessity of copying the fitness structure to the CPU on each iteration. GPU-based move selection eliminates this data transfer, but an efficient selection algorithm needs to be in place on the GPU. A clear example is simple steepest descent, where the best move can be computed by a standard reduction operation. A tabu search can also be implemented on the GPU by first checking for each move whether it is tabu and then reducing to the best non-tabu move. In general, it may not be clear which approach will perform better; it depends on the situation at hand. In such cases, the alternative implementations must be compared. All routing-related papers we found use either one or the other approach for a given algorithm, see Table 5. Luong et al. (2011b) compare them for hill climbing on the permuted perceptron problem.

If move selection is performed on the GPU, the update of the current solution may also be performed on the device. This eliminates the otherwise necessary copying of the updated current solution from the CPU to the GPU. Alternatively, the chosen move can be copied to the GPU (Coelho et al. 2012).

Efficiency aspects and limitations of local search on the GPU

In CUDA it is not possible to synchronize between blocks inside a kernel. Since most papers employ a one-move-per-thread approach, the LS process needs to be implemented using several kernels. In combination with the different copy operations that might be needed, the question of asynchronous execution becomes important. By using streams in combination with asynchronous CPU–GPU coordination, it is possible to reduce the time where the GPU is idle, even to zero. Only the paper by Schulz (2013) proposes and investigates an asynchronous execution pattern.

The efficiency of a kernel is obviously important for the overall speed of the computation. The papers (Luong et al. 2011b; O’Neil et al. 2011; Coelho et al. 2012; Rocki and Suda 2012; Schulz 2013) all discuss some implementation details and CUDA-specific optimizations. Only Schulz (2013) provides a profiling analysis of the presented details.

So far we have assumed that the GPU memory is large enough to store all necessary information such as problem data, the current solution, and the fitness structure. For very large neighborhoods the fitness structure might not fit into GPU memory. Luong et al. (2011b) mention this problem. They seem to solve it by assigning several moves to one thread. Schulz (2013) provides an implementation for very large neighborhoods by splitting the neighborhood in parts.

When evaluating the whole neighborhood one naturally selects a single, best improving move. However, as observed by Burke and Riise (2012), one may waste

a lot of computational effort. They suggest an alternative strategy where one finds independent improving moves and applies them all. This reduces the amount of iterations needed to find a local optimum.

Multi-start Local Search

Pure local search is guaranteed to get stuck in a local optimum, given sufficient time. Amongst alternative remedies, multi-start LS maybe the simplest. New initial solutions may be generated randomly, or with management of diversity. Multi-start LS thus provides another degree of parallelism, where one local search instance is independent of the other. In the GPU literature we have found two main approaches. Either, a GPU-based parallel neighborhood evaluation of the different local searches is performed sequentially (Luong et al. 2011a), or the local searches run in parallel on the GPU (O'Neil et al. 2011; Zhu et al. 2010; Luong et al. 2011a).

For approaches where there is no need for data transfer between the CPU and GPU during LS, the former scheme should be able to keep the GPU fully occupied with neighborhood evaluation. However, LS might use a complicated selection procedure that is more efficient to execute on the CPU, despite the necessary copy of fitness structure. In this case one could argue that using sequential parallel neighborhood evaluation will lead to too many CPU-GPU copy operations, slowing down the overall algorithm. However, this is not necessarily true. If the copying of data takes less time than the neighborhood evaluation, asynchronous execution might be able to fully hide the data transfer. In one iteration, while the fitness structure of the i th local search is copied to the CPU, the GPU can already evaluate the neighborhood for the next, j th local search where $j = i + 1$. Once the copying is finished, the CPU can then perform move selection for the i th local search, all while the GPU is still evaluating the neighborhood of the j th local search.

The second idea of using one thread per LS instance also has its drawbacks. First, for the GPU to be fully utilized, thousands of threads are needed. This raises the question, whether, from a solution quality point of view, it makes sense to have that many local searches. On the GPU, all threads in a warp perform exactly the same operation at any time. Hence, all local searches in a warp must use the same type of neighborhood. Moreover, different local searches in a warp might have widely varying numbers of iterations until they reach a local optimum. If all threads in the same warp simply run their local search to the end, they have to 'wait' until the last of their local searches is finished before the warp can be destroyed.

There are ways to tackle these problems. O'Neil et al. (2011) use the same neighborhood for all local searches and employ a kind of load balancing to avoid threads within a warp waiting for the others to complete. Another idea, used, e.g. in (Zhu et al. 2010; Luong et al. 2011a) is to let the LS in each thread run only for a given number of iterations and then perform restart or load balancing before continuing. Due to the many variables involved, it is impossible to state generally that the sequential parallel neighborhood evaluation is better or worse than the one thread per local search approach. Even for a given situation, such a statement needs to be based on implementations that have been thoroughly optimized, analyzed, and profiled, so that the advantages and limitations of each approach become apparent.

We have not found a paper that provides such a thorough comparison between the two approaches.

GPU computing for shortest path problems

Already in 2004, Micikevicius (2004) describes his graphics pipeline GPU implementation of the Warshall–Floyd algorithm for the all-pairs shortest paths problem. He reports speedups of up to $3\times$ over a CPU implementation. In 2007, Harish and Narayanan (2007) utilize CUDA to implement breadth first search, single source shortest path, and all-pairs shortest path algorithms aimed at large graphs. They report speedups, but point out that the size of the device memory limits the size of the graphs handled on a single GPU. Also, the GPU at the time only supported single precision arithmetic. Katz and Kider (2008) describe a shared memory cache efficient CUDA implementation to solve transitive closure and the all-pairs shortest-path problem on directed graphs for large datasets. They report good speedups both on synthetic and real data. In contrast with the implementation of Harish and Narayanan, the graph size is not limited by the device memory.

Buluç et al. (2010) implemented (CUDA) a recursively partitioned all-pairs shortest-paths algorithm where almost all operations are cast as matrix-matrix multiplications on a semiring. They report that their implementation runs more than two orders of magnitude faster on an NVIDIA 8800 GPU than on an Opteron CPU. The number of vertices in the test graphs used vary between 512 and 8192. The all-pairs SPP was also studied by Tran (2010), who utilized CUDA to implement two GPU-based algorithms and reports an incredible speedup factor of 2,500 relative to a single core implementation.

In a recent paper Delling et al. (2011) present a novel algorithm called PHAST to solve the nonnegative single-source SPP on road networks and other graphs with low highway dimension. PHAST takes advantage of features of modern CPU architectures, such as SSE and multi-core. According to the authors, the method needs fewer operations, has better locality, and is better able to exploit parallelism at multicore and instruction levels when compared to Dijkstra's algorithm. They also implement a GPU version of PHAST (GPHAST) with CUDA, and report up to three orders of magnitude speedup relative to Dijkstra's algorithm on a high-end CPU. They conclude that GPHAST enables practical all-pairs shortest-paths calculations for continental-sized road networks.

With robotics applications as main focus, Kider et al. (2010) implement a GPU version of R^* , a randomized, non-exact version of the A^* algorithm, called R^* GPU. They report that R^* GPU consistently produces lower cost solutions, scales better in terms of memory, and runs faster than R^* .

Literature on non-routing problems

Although the specifics of a metaheuristic may change according to the problem at hand, its main idea stays the same. Therefore, it is also interesting to study GPU implementations of metaheuristics in a non-routing setting. This is especially true

for metaheuristics where so far no routing-related GPU implementation exists. In the following, we present a short overview over existing GPU literature for metaheuristics applied to DOPs other than routing problems.

Swarm intelligence metaheuristics (non-ACO, non-routing)

Particle swarm optimization is normally considered to belong to swarm intelligence methods, but may also be regarded as a population based method. Just as GA, PSO may be used both for continuous and DOPs. An early PSO on GPU paper is Li et al. (2007). They use the graphics pipeline for fine-grained parallelization of PSO and perform computational experiments on three unconstrained continuous optimization problems. Speedup factors up to 5.7 were observed. In 2011, Solomon et al. (2011) report from an implementation of a collaborative multi-swarm PSO algorithm on the GPU for a real-life DOP application: the task matching problem in a heterogeneous distributed computing environment. They report speedup factors of up to 37.

Emergent behavior in biology, e.g., flocking birds and schooling fish, was an inspiration for PSO. However, the flocking birds brand is still used for PSO-like swarm intelligence methods in optimization. Charles et al. (2008) study flocking-based document clustering on the GPU and report a speedup of 3–5 relative to a CPU implementation. In a 2011 follow-up paper with partly the same authors (Cui et al. 2011), speedup factors of 30–60 were observed. In an undergraduate honors thesis, Weiss (2010) investigates GPU implementation of two special purpose swarm intelligence algorithms for data mining: an ACO algorithm for rule-based classification, and a bird-flocking algorithm for data clustering. He concludes that the GPU implementation provides significant benefits.

Population-based metaheuristics (non-routing)

Yu et al. (2005) describe an early (2005) implementation of a fine-grained parallel genetic algorithm for continuous optimization, referring to the 1991 paper by Spiessens and Manderick (1991) on massively parallel GA. They were probably the first to design and implement a GA on the GPU, using the graphics pipeline. Their approach stores chromosomes and their fitness values in the GPU texture memory. Using the Cg language for the graphics pipeline, fitness evaluation and genetic operations are implemented entirely with fragment programs (shaders) that are executed on the GPU in parallel. Performance of an NVidia GeForce 6800GT GPU implementation was measured and compared with a sequential AMD Athlon 2500+ CPU implementation. The Colville function in unconstrained global optimization was used as benchmark. For genetic operators, the authors report speedups between 1.4 (population size 32^2) and 20.1 (population size 512^2). Corresponding speedups for fitness evaluation are 0.3 and 17.1, respectively.

Also in 2005, Luo et al. (2005) describe their use of the graphics pipeline and the Cg language for a parallel genetic algorithm solver for 3-SAT. They compare performance between two hardware platforms.

Wong et al. (2005), Wong and Wong (2006) and Fok et al. (2007) investigate hybrid computing GAs where population evaluation and mutation are performed on

the GPU, but the remainder is executed on the CPU. Wong (2009) extends the work to multi-objective GAs and uses CUDA for the implementation. For a recent comprehensive survey on GPU computing for EA and GA, but not including Genetic Programming, see Section 1.3.2 of the PhD Thesis of Luong (2011).

Genetic programming (GP) is a special application of GA where each individual is a computer program. The overall goal is automatic programming. Early GPU implementations (2007) are described by Chitty (2007), who uses the graphics pipeline and Cg. Harding and Banzhaf (2007b) also use the graphics pipeline but with the Accelerator package, a .Net assembly that provides access to the GPU via DirectX. Several papers (Harding and Banzhaf 2007a, 2011; Langdon and Banzhaf 2007, 2008; Banzhaf et al. 2008; Langdon and Harrison 2008) report from extensions of this initial work. Robilliard et al. (2008, 2009a, b) have published three papers on GPU-based GP using CUDA, initially with a fine-grained parallelization scheme on the G80 GPU, then with different parallelization schemes and better speedups. Maitre et al. (2010) report from similar work. For details, we refer to the recent survey by Langdon (2011) and the individual technical papers.

Local search and trajectory-based metaheuristics (non-routing)

Luong et al. have published several follow-up papers to Luong et al. (2009, 2011a, b). In (Luong et al. 2010a) they discuss how to implement LS algorithms with large-size neighborhoods on the GPU², with focus on memory issues. Their general design is based on so-called iteration-level parallelization, where the CPU manages the sequential LS iterations, and the GPU is dedicated to parallel generation and evaluation of neighborhoods. Mappings between threads and neighbors are proposed for LS operators with Hamming distance 1, 2, and 3. From an experimental study on instances of the permuted perceptron problem from cryptography the authors conclude that speedup increases with increasing neighborhood cardinality (Hamming distance of the operator) and that the GPU enables the use of neighborhood operators with higher cardinality in LS. Similar reports are found in Luong et al. (2010b, c). The PhD thesis of Luong from 2011 (Luong 2011) contains a general discussion on GPU implementation of metaheuristics, including results from the papers mentioned above.

The paper by Janiak et al. (2008) applies tabu search also to the permutation flowshop scheduling problem (PFSP) with the Makespan criterion. Their work on the PFSP was continued by Czapiński and Barnes (2011). They describe a tabu search metaheuristic based on swap moves. The GPU implementation was done with CUDA. Two implementations of move selection and tabu list management were considered. Performance was optimized through experiments and tuning of several implementation parameters. Good speedups were reported, both relative to the GPU implementation of Janiak et al. and relative to a serial CPU implementation, for randomly generated PFSP instances with 10–500 tasks and 5–30

² The title of the paper may suggest that it discusses the large neighborhood search metaheuristic, but this is not the case.

machines. The authors mainly attribute the improved efficiency over Janiak et al. to better memory management.

The first of three publications we have found on GPU implementation of Simulated annealing (SA) is a conference paper by Choong et al. (2010). SA is the preferred method for optimization of FPGA placement³. Han et al. (2011) study SA on the GPU for IC floorplanning by using CUDA. They work with multiple solutions in parallel and evaluate several moves per solution in each iteration. As the GPU-based algorithm works differently than the CPU method, Han et al. examine three different modifications to their first GPU implementation with respect to solution quality and speedup. They achieve a speedup of up to 160 for the best solution quality, where the computation times are compared with the CPU code from the UMPack suite of VLSI-CAD tools (Adya and Markov 2003). Stivala et al. use GPU-based SA in (Stivala et al. 2010) for the problem of searching a database for protein structures or occurrences of substructures. They develop a new SA-based algorithm for the given problem and provide both a CPU and a GPU implementation⁴. Each thread block in the GPU version runs its own SA schedule, where the threads perform the database comparisons. The quality of the proposed method varies with different problems, but good speedups of the GPU version versus the CPU one are obtained.

Hybrid metaheuristics

The definition of *hybrid metaheuristics* may seem unclear. In the literature, it often refers to methods where metaheuristics collaborate or are integrated with exact optimization methods from mathematical programming, the latter also known as *matheuristics*. A restricted definition to combinations of different metaheuristics arguably has diminishing interest, as increasing emphasis in the design of modern metaheuristics is put on the combination and extension of relevant working mechanisms of different classical metaheuristics. As regards hybrid methods, the three relevant publications we have found all discuss GPU implementation of combinations of genetic algorithms with LS, a basic form of *memetic algorithms*.

In 2006, Luo and Liu (2006) follow up on the 2005 graphics pipeline GA paper on the 3-SAT problem by Luo et al. (2005) referred to in “[Population based metaheuristics \(non-routing\)](#)” above. They develop a modified version of the parallel CGWSAT hybrid of cellular GA and greedy local search due to Folino et al. (1998) and implement it on a GPU using the graphics pipeline with Cg. They report good speedups over a CPU implementation with similar solution quality. GPU-based hybrids of GA and LS for Max-SAT were investigated in 2009 by Munawar et al. (2009).

Krüger et al. (2010) present the first implementation of a generic memetic algorithm for continuous optimization problems on a GTX295 gaming card using CUDA. Reportedly, experiments on the Rosenbrock function and a real-world problem show speedup factors between 70 and 120.

³ As discussed in “[Early works on non-GPU related accelerators](#)” above, FPGAs were used in early works in heterogeneous discrete optimization.

⁴ The CPU version is generated by compiling the kernels for the CPU.

Luong et al. (2012) propose a load balancing scheme to distribute multiple metaheuristics over both the GPU and the CPU cores simultaneously. They apply the scheme to the quadratic assignment problem using the fast ant metaheuristic, yielding a combined speedup (both multiple cores on CPU and GPU) of up to 15.8 compared with a single core on the CPU.

GPU implementations of Linear Programming and Branch & Bound

Also relevant to discrete optimization we found five publications on GPU implementation of linear programming (LP) methods. Greeff (2005) published a technical report on a GPU graphics pipeline implementation of the revised simplex method in 2005. Reported speedups were large compared with a CPU implementation. The implementation could not solve problems with more than 200 variables, however.

In their 2008 paper, Jung and O'Leary (2008) present a mixed-precision CPU-GPU interior point LP algorithm. By comparing GPU and CPU implementations, they demonstrated performance improvement for sufficiently large dense problems with up to some 700 variables and 500 constraints.

In 2009, Spampinato and Elster (2009) published a continuation of the work by Greeff from 2005. Their CUDA implementation of the revised simplex method solves LPs with up to 2000 variables on a CPU/GPU system. They report speedups factors of 2.5 for large problem instances.

Early GPUs had only single precision arithmetic. In 2011, Lalami et al. (2011b) report a maximum speedup of 12.5 for their simplex method implementation with double precision arithmetic on a GTX 260 GPU. They use randomly generated non-sparse LP instances. Also in 2011, the same authors report from a CUDA implementation of the simplex method on a multi GPU architecture (Lalami et al. 2011a). Computational tests on random, non-sparse instances show a maximum speedup of 24.5 with two Tesla C2050.

Branch & Bound is a widely used exact method for solving DOPs. Chakroun et al. (2012) use the GPU for the bound operator in the algorithm applied to the flow shop scheduling problem. The paper discusses GPU-specific details of the implementation and in experiments a speedup of up to 77.5 compared with a single core on a CPU is achieved.

Lessons for future research

In the previous section we presented a literature survey on GPU computing in discrete optimization and a more detailed discussion of selected papers on routing problems. In the following we will provide our views on future research on GPU computing in discrete optimization.

GPU implementations in discrete optimization

The overwhelming majority of routing-related papers on GPU usage in discrete optimization has focused on relatively simple, well-known optimization algorithms

on the GPU. A main goal is to compare GPU implementations with equivalent single core CPU versions. The results predominantly show significant speedups and hence provide proofs of concept. The observations are consistent with GPU-related research from other parts of scientific computing. Also in optimization, the GPU is a viable and powerful tool that can be used to increase performance. This is not uninteresting, particularly from a pragmatic stance. Also from a scientific point of view, proof of concept papers are important. More power for computational experiments will lead to better algorithms and better understanding of optimization problems.

Is this the final word? Far from it. Most of the relevant literature does not consider important aspects of GPU usage and the development of *novel* algorithms which fully utilize the combined advantages of the CPU and the GPU to provide faster and more robust solutions. In our opinion, the subfield of GPU computing in discrete optimization is still in its infancy.

For a practitioner it may be of little interest whether the GPU or CPU is used to its full capacity. From a scientific perspective we would like to use scientific methods to develop algorithms which are able to yield better and more robust solutions than the algorithms of today by fully utilizing all available hardware efficiently. To achieve this goal, research that provides knowledge and ideas towards this end is welcome. What qualifies such research, and what is lacking so far?

Focusing on comparing CPU and GPU versions of an algorithm is an important step to provide proof of concept implementations showing the performance potential provided by the GPU. Nevertheless, towards the specified scientific goal of new and efficient algorithms, this approach has several potential drawbacks.

Solution quality Many of the papers comparing a CPU and a GPU implementation do not discuss solution quality. On the one hand, if the algorithm is the same, it can be expected that the solution quality is too. However, the considered algorithm that is run on the GPU might not be a state-of-the-art CPU-based algorithm and thus not be competitive in terms of latest solution quality.

CPU speed Similar to the point above, the used algorithm might not be cutting edge for the CPU. Hence, even if the GPU implementation is faster than its CPU counterpart, the leading CPU algorithm might still be faster than the studied GPU implementation. In addition, the considered implementation of the algorithm on the CPU might not be state-of-the-art. An efficient GPU implementation requires effort in finding the right memory access patterns, the right distribution of data over the different memories, synchronization and cooperation strategies, and much more. An equally optimized CPU implementation would amongst others utilize multiple cores, have caching strategies and use SSE or AVX instructions⁵. Such an effort is rarely seen in the literature.

⁵ Modern CPUs support vector operations, enabling simultaneous operations on all elements of those vectors (Fog 2013). These so-called SIMD extensions/operations started with MMX on 64byte registers and developed further with SSE (128byte registers) into AVX (256byte registers). For a coarse overview see (Wikipedia 2013), a more detailed discussion of the operations including examples can be found in (Fog 2013).

GPU usage Although the GPU implementation might perform faster than the CPU implementation, it does not mean it uses the GPU efficiently. There might be a better way to distribute the work over the GPU architecture, a faster memory access pattern, or other improving variants. It might be that the GPU implementation is using the GPU only a fraction of the time, leaving it idle for a substantial part of the time. This means that there could be a different implementation or algorithm for the problem which is able to use the GPU more efficiently, with resulting speed and/or quality improvement.

CPU usage In most of the papers comparing CPU and GPU implementations, the CPU is basically idle the whole time. This is a waste of computational resources. A truly heterogeneous algorithm will typically have higher performance.

In our opinion, future research papers on GPU usage in discrete optimization should contain algorithm analysis and analysis of hardware utilization. Such analyses will identify areas of further improvement, spawn ideas for novel algorithms, and point to further research directions. Such analyses are time consuming. Although the potential gain is high⁶, one cannot expect that researchers in optimization will follow these steps of research in computational science to their end. We think that initial steps should be mandatory, however.

Algorithm analysis

This is obviously a wide area that covers mathematical analyses as well as computational experiments. Such analyses may show that a known algorithm, deemed too inefficient on the CPU, can now be used beneficially⁷ with the help of the GPU. Another example is the development of new algorithms that use the intrinsic properties of the available hardware (CPU and GPU together) to provide better or more robust solutions. Clearly one focus here would be on the improvement of the solution quality. In general, when studying algorithms on the GPU, one has to make sure that the work done on the GPU is actually beneficial to the algorithm. In LS one could, for example, question the meaning of evaluating billions of moves if just one of them is applied afterwards. Does this really increase the solution quality compared with a simpler first improvement strategy? One could, as suggested by Burke and Riise (2012), utilize several of the improving moves found.

Hardware utilization

Hardware utilization should be analyzed, at least to a basic level, so major bottlenecks are identified and removed. This includes an examination of the CPU–GPU coordination and whether asynchronous execution patterns might be possible

⁶ The paper by Schulz (2013) indicates an order of magnitude speedup by careful tuning of a basic GPU implementation.

⁷ Beneficially here means to improve the overall solution quality, speed or robustness of the overall solution method.

and beneficial. An example is found in the paper by Schulz (2013), although in general it will not be possible to conduct such a detailed and time-consuming analysis and performance tuning. The analysis and conclusions should be based on solid scientific methods and fair comparison.

Even if it is not possible to perform the final steps of performance optimization, it is important to understand whether an algorithm or implementation is able to use the hardware efficiently. If not, it is equally interesting to discover why this is not the case and what the limiting factors are. This will provide valuable information for the development of other, more efficient algorithms or implementation approaches.

Heterogeneous discrete optimization in general

The lessons learnt from GPU-based algorithms in discrete optimization are in principle also true for heterogeneous discrete optimization. The goal should be algorithms that use all available hardware resources⁸ efficiently towards finding high-quality solutions. Ideally, such algorithms should be self-adapting and automatically configure themselves to the problem, the hardware, and even to the problem-solving status while executing. We think that papers in heterogeneous discrete optimization and similar areas should give a reasonable contribution in the form of knowledge that can be used to create and develop such algorithms. This requires full specification of hardware platforms utilized as well as algorithmic and implementational details.

A promising and virtually unexplored research avenue is the development of collaborative methods in discrete optimization that fully utilize modern, heterogeneous PC architectures. In the next ten years we may see a general performance increase in discrete optimization that surpasses the historical increase pointed to by Bixby (2002) for commercial LP solvers.

Summary and conclusion

The sequence of two papers of which this paper is the second, has two primary goals. The first, addressed in Part I (Brodtkorb et al. 2013), is to provide a tutorial style introduction to modern PC architectures and the computational performance increase opportunities that they offer through a combination of parallel cores for task parallelization and one or more stream processing accelerators. The second goal, addressed in Part II here, is to present a survey of the literature relevant to discrete optimization and routing problems in particular.

Part I (Brodtkorb et al. 2013) starts with a short overview of the historical development of CPUs and stream processing accelerators such as the GPU, followed by a brief discussion of the development of more user-friendly GPU programming environments. To illustrate modern GPU programming with CUDA, we provided a concrete example: local search for the TSP. This was followed by the

⁸ I.e., multiple CPU cores and one or more stream processing accelerators according to the scope of this paper.

presentation of best practice and state-of-the-art strategies for developing efficient GPU code. We also discussed heterogeneous aspects involved in keeping both the CPU and the GPU busy. Here, in Part II, we provide a comprehensive survey of the existing literature on parallel discrete optimization for modern PC architectures with focus on routing problems. Virtually all related papers report on implementation of an existing optimization algorithm on a stream processing accelerator, mostly the GPU. We provide a critical, detailed review of the literature relevant to routing problems. Finally, we present lessons learnt and our subjective views on future research directions.

GPU computing in discrete optimization is still in its infancy. The bulk of the literature consists of reports from rather basic implementations of existing optimization methods on GPU, with measurement of speedup relative to a CPU implementation of unknown quality. It is our opinion that further research should be performed in a more scientific fashion: with stronger focus on the efficiency of the implementation, proper analyses of algorithms and hardware utilization, thorough and fair measurement of speedup, with efforts to utilize all of the available hardware, and with reports that better enable reproduction. The ultimate goal would be the development of novel, fast, and robust high-quality methods that exploit the full heterogeneity of modern PCs efficiently while at the same time being flexible by self-adapting to the hardware at hand. The potential gains are hard to over-estimate.

Acknowledgments The work presented in this paper has been partially funded by the Research Council of Norway as a part of the Collab project (contract number 192905/I40, SMARTRANS), the DOMinant II project (contract number 205298/V30, eVita), the Respons project (contract number 187293/I40, SMARTRANS), and the CloudViz project (contract number 201447, VERDIKT).

References

- Aarts E, Lenstra JK (eds) (2003) Local search in combinatorial optimization. Princeton University Press, Oxford
- Acan A (2002) G A A C O: A G A + A C O hybrid for faster and better search capability. In: Dorigo M, Di Caro G, Sampels M (eds) Ant algorithms. Lecture notes in computer science. In: Proceedings of Third International Workshop, ANTS 2002, vol 2463. Springer, Berlin, pp 300–301
- Adya S, Markov I (2003) Fixed-outline floorplanning: enabling hierarchical design. IEEE transactions on very large scale integration (VLSI) systems, vol 11, no. 6, pp 1120–1135
- Alba E (2005) Parallel metaheuristics: a new class of algorithms. In: Wiley series on parallel and distributed computing. Wiley, New York
- Bai H, OuYang D, Li X, He L, Yu H (2009) MAX–MIN snt system on GPU with CUDA. In: Fourth international conference on innovative computing, information and control (ICICIC), pp 801–804
- Banzhaf W, Harding S, Langdon W, Wilson G (2008) Accelerating genetic programming through graphics processing units. In: Riolo RL, Soule T, Worzel B (eds) Genetic programming theory and practice, vol VI. Springer, Berlin, pp 229–249
- Bixby RE (2002) Solving real-world linear programs: a decade and more of progress. Oper Res 50:3–15
- Bleiweiss A (2008) GPU accelerated pathfinding. In: Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS symposium on graphics hardware, GH '08, pp 65–74. Eurographics Association, Aire-la-Ville, Switzerland
- Brodtkorb AR, Hagen TR, Schulz C, Hasle G (2013) GPU computing in discrete optimization—part I: introduction to the GPU. EURO J Transp Logist. doi:[10.1007/s13676-013-0025-1](https://doi.org/10.1007/s13676-013-0025-1)
- Buluç A, Gilbert JR, Budak C (2010) Solving path problems on the GPU. Parallel Comput 36(5–6):241–253

- Bura W, Boryczka M (2011) The parallel ant vehicle navigation system with CUDA technology. In: Jedrzejowicz P, Nguyen N, Hoang K (eds) Computational collective intelligence. Technologies and applications. Lecture notes in computer science, vol 6923. Springer, Berlin, pp 505–514
- Burke EK, Riise A (2012) On parallel local search for permutations (Submitted)
- Catala A, Jaen J, Modiolí J (2007) Strategies for accelerating ant colony optimization algorithms on graphical processing units. In: 2007 IEEE congress on evolutionary computation (CEC 2007), pp 492–500
- Cecilia J, Garcia J, Ujaldon M, Nisbet A, Amos M (2011) Parallelization strategies for ant colony optimisation on GPUs. In: 2011 IEEE international symposium on parallel and distributed processing workshops and PhD forum (IPDPSW), pp 339–346 (2011)
- Chakroun I, Mezmaš M, Melab N, Bendjoudi A (2012) Reducing thread divergence in a GPU-accelerated branch-and-bound algorithm. *Concurr Comput Pract Exp*. doi:[10.1002/cpe.2931](https://doi.org/10.1002/cpe.2931)
- Charles J, Potok T, Patton R, Cui X (2008) Flocking-based document clustering on the graphics processing unit. In: Krasnogor N, Nicosia G, Pavone M, Pelta D (eds) Nature inspired cooperative strategies for optimization (NICSO 2007). Studies in computational intelligence, vol 129, pp 27–37. Springer, Berlin
- Chen S, Davis S, Jiang H, Novobilski A (2011) CUDA-based genetic algorithm on traveling salesman problem. In: Lee R (ed) Computer and information science 2011. Studies in computational intelligence, vol 364. Springer, Berlin, pp 241–252
- Chitty DM (2007) A data parallel approach to genetic programming using programmable graphics hardware. In: Thierens D, Beyer HG, Bongard J, Branke J, Clark JA, Cliff D, Congdon CB, Deb K, Doerr B, Kovacs T, Kumar S, Miller JF, Moore J, Neumann F, Pelikan M, Poli R, Sastry K, Stanley KO, Stutzle T, Watson RA, Wegener I (eds) GECCO '07: proceedings of the 9th annual conference on genetic and evolutionary computation, vol 2. ACM Press, London, pp 1566–1573
- Choong A, Beidas R, Zhu J (2010) Parallelizing simulated annealing-based placement using GPGPU. In: 2010 international conference on field programmable logic and applications (FPL), pp 31–34
- Coelho I, Ochi L, Munhoz P, Souza M, Farias R, Bentes C (2012) The single vehicle routing problem with deliveries and selective pickups in a CPU-GPU heterogeneous environment. In: 2012 IEEE 14th international conference on high performance computing and communication, pp 1606–1611
- Crainic TG, Le Cun B, Roucairol C (2006) Parallel Branch-and-Bound algorithms. Wiley, New York, pp 1–28
- Crainic TG (2008) Parallel solution methods for vehicle routing problems. In: Golden B, Raghavan S, Wasil E, Sharda R, Voss S (eds) The vehicle routing problem: latest advances and new challenges. Operations research/computer science interfaces series, vol 43. Springer, New York, pp 171–198
- Cui X, St. Charles J, Beaver J, Potok T (2011) The GPU enhanced parallel computing for large scale data clustering. In: 2011 international conference on cyber-enabled distributed computing and knowledge discovery (CyberC), pp 220–225
- Czapiński M, Barnes S (2011) Tabu search with two approaches to parallel flowshop evaluation on CUDA platform. *J Parallel Distrib Comput* 71:802–811
- Delévacq A, Delisle P, Gravel M, Krajecki M (2013) Parallel ant colony optimization on graphics processing units. *Metaheuristics on GPUs*. *J Parallel Distrib Comput* 73(1): 52–61
- Delling D, Goldberg AV, Nowatzyk A, Werneck RF (2011) PHAST: hardware-accelerated shortest path trees. In: Proceedings of the 2011 IEEE international parallel & distributed processing symposium, IPDPS '11. IEEE Computer Society, Washington, DC, pp 921–931
- Diego FJ, Gómez EM, Ortega-Mier M, García-Sánchez A (2012) Parallel CUDA architecture for solving de VRP with ACO. In: Sethi SP, Bogataj M, Ros-McDonnell L (eds) Industrial engineering: innovative networks. Springer, London, pp 385–393
- Dorigo M, Stützle T (2004) Ant colony optimization. Bradford Company, Scituate
- Fog A (2013) Optimizing software in C++—an optimization guide for windows, linux and Mac platforms, Copenhagen University College of Engineering. <http://www.agner.org/optimize>
- Fok KL, Wong TT, Wong ML (2007) Evolutionary computing on consumer graphics hardware. *Intell Syst IEEE* 22(2):69–78
- Folino G, Pizzuti C, Spezzano G (1998) Solving the satisfiability problem by a parallel cellular genetic algorithm. In: Proceedings of Euromicro workshop on computational intelligence, IEEE computer, Society Press, pp 715–722
- Fujimoto N, Tsutsui S (2011) A highly-parallel TSP solver for a GPU computing platform. In: Dimov I, Dimova S, Kolkovska N (eds) Numerical methods and applications. Lecture notes in computer science, vol 6046. Springer, Berlin, pp 264–271

- Greeff G (2005) The revised simplex algorithm on a GPU. Technical report, University of Stellenbosch
- Guntsch M, Middendorf M, Scheuermann B, Diessel O, ElGindy H, Schmeck H, So K (2002) Population based ant colony optimization on FPGA. In: 2002 IEEE international conference on field-programmable technology, 2002 (FPT). Proceedings, pp 125–132
- Han Y, Roy S, Chakraborty K (2011) Optimizing simulated annealing on GPU: a case study with IC floorplanning. In: 12th international symposium on quality electronic design (ISQED), 2011, pp 1–7
- Harding S, Banzhaf W (2007a) Fast genetic programming on GPUs. In: Ebner M, O'Neill M, Ekárt A, Vanneschi L, Esparcia-Alcázar AI (eds) Proceedings of the 10th European conference on genetic programming. Lecture notes in computer science, vol 4445. Springer, Valencia, pp 90–101
- Harding SL, Banzhaf W (2007b) Fast genetic programming and artificial developmental systems on GPUs. In: 21st international symposium on high performance computing systems and applications (HPCS'07). IEEE Computer Society, Canada, p 2
- Harding S, Banzhaf W (2011) Implementing cartesian genetic programming classifiers on graphics processing units using GPU.NET. In: Harding S, Langdon WB, Wong ML, Wilson G, Lewis T (eds) GECCO 2011 computational intelligence on consumer games and graphics hardware CIGPU. ACM, New York, pp 463–470
- Harish P, Narayanan PJ (2007) Accelerating large graph algorithms on the GPU using CUDA. In: Proceedings of the 14th international conference on High performance computing, HiPC'07. Springer, Berlin, pp 197–208
- Janiak A, Janiak W, Lichtenstein M (2008) Tabu search on GPU. *J Univers Comput Sci* 14(14): 2416–2427
- Katz GJ, Kider Jr JT (2008) All-pairs shortest-paths for large graphs on the GPU. In: Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS symposium on graphics hardware, GH '08. Eurographics Association, Aire-la-Ville, pp 47–55
- Kennedy J, Eberhart R, Shi Y (2001) Swarm intelligence. In: The Morgan Kaufmann series in evolutionary computation. Morgan Kaufmann Publishers, Burlington
- Kider J, Henderson M, Likhachev M, Safonova A (2010) High-dimensional planning on the GPU. In: 2010 IEEE international conference on robotics and automation (ICRA), pp 2515–2522
- Krüger F, Maitre O, Jimenez S, Baumes L, Collet P (2010) Speedups between $\times 70$ and $\times 120$ for a generic local search (memetic) algorithm on a single GPGPU chip. In: Di Chio C, Cagnoni S, Cotta C, Ebner M, Ekárt A, Esparcia-Alcazar A, Goh CK, Merelo J, Neri F, Preu M, Togelius J, Yannakakis G (eds) EvoNum 2010, LNCS, vol 6024. Springer, Berlin, pp 501–511
- Lalami M, El-Baz D, Boyer V (2011a) Multi GPU implementation of the simplex algorithm. In: 2011 IEEE 13th international conference on high performance computing and communications (HPCC), pp 179–186
- Lalami ME, Boyer V, El-Baz D (2011b) Efficient implementation of the simplex method on a CPU-GPU system. In: Proceedings of the 2011 IEEE international symposium on parallel and distributed processing workshops and PhD forum, IPDPSW '11. IEEE Computer Society, Washington, DC, pp 1999–2006
- Langdon W, Banzhaf W (2007) A SIMD interpreter for genetic programming on GPU graphics cards. In: O'Neill M, Vanneschi L, Gustafson S, Esparcia Alcazar AI, De Falco I, Della Cioppa A, Tarantino E (eds) Proceedings of the 11th European conference on genetic programming, EuroGP 2008. Springer, Berlin, pp 73–85
- Langdon W, Harrison A (2008) GP on SPMD parallel graphics hardware for mega bioinformatics data mining. *Soft Comput Fusion Found Methodol Appl* 12(12):1169–1183
- Langdon WB (2011) Graphics processing units and genetic programming: an overview. *Soft Comput* 15:1657–1669
- Li J, Wan D, Chi Z, Hu X (2007) An efficient fine-grained parallel particle swarm optimization method based on GPU-acceleration. *Int J Innov Comput Inf Control* 3(6):1707–1714
- Li J, Chi Z, Wan D (2008) Parallel genetic algorithm based on fine-grained model with GPU-accelerated. *Control Decis* 23(6)
- Li J, Hu X, Pang Z, Qian K (2009a) A parallel ant colony optimization algorithm based on fine-grained model with GPU acceleration. *Int J Innov Comput Inf Control* 5(11(A)):3707–3716
- Li J, Zhang L, Liu L (2009b) A parallel immune algorithm based on fine-grained model with GPU-acceleration. In: Proceedings of the 2009 fourth international conference on innovative computing, information and control, ICICIC '09. IEEE Computer Society, Washington, DC, pp 683–686
- Luo Z, Yang Z, Liu H, Lv W (2005) GA computation of 3-SAT problem on graphic process unit. *Environ Bioindic* 1:7–11

- Luo Z, Liu H (2006) Cellular genetic algorithms and local search for 3-SAT problem on graphic hardware. In: IEEE congress on evolutionary computation, CEC 2006, pp 2988–2992
- Luong TV, Melab N, Talbi EG (2009) Parallel local search on GPU. Rapport de recherche RR-6915, INRIA
- Luong TV, Melab N, Talbi EG (2010a) Large neighborhood local search optimization on graphics processing units. In: 2010 IEEE international symposium on parallel distributed processing, workshops and Phd forum (IPDPSW), pp 1–8
- Luong TV, Melab N, Talbi EG (2010b) Local search algorithms on graphics processing units. A case study: the permutation perceptron problem. In: EvoCOP, pp 264–275
- Luong TV, Melab N, Talbi EG (2010c) Neighborhood structures for GPU-based local search algorithms. *Parallel Process Lett* 20(4):307–324
- Luong TV (2011) Métaheuristiques parallèles sur GPU. Ph.D. thesis, Université des Sciences et Technologie de Lille-Lille I (This thesis is written in English)
- Luong TV, Melab N, Talbi EG (2011a) GPU-based multi-start local search algorithms. In: Coello C (ed) Learning and intelligent optimization. Lecture notes in computer science, vol 6683. Springer, Berlin, pp 321–335
- Luong TV, Melab N, Talbi EG (2011b) GPU computing for parallel local search metaheuristic algorithms. *IEEE Trans Comput* 99(PrePrints). <http://doi.ieeecomputersociety.org/10.1109/TC.2011.206>
- Luong TV, Taillard E, Melab N, Talbi EG (2012) Parallelization strategies for hybrid metaheuristics using a single GPU and multi-core resources. In: Coello C, Cutello V, Deb K, Forrest S, Nicosia G, Pavone M (eds) Parallel problem solving from nature-PPSN XII. Lecture notes in computer science, vol 7492. Springer, Berlin, pp 368–377
- Maitre O, Lachiche N, Collet P (2010) Fast evaluation of GP trees on GPGPU by optimizing hardware scheduling. In: Proceedings of the 13th European conference on genetic programming, EuroGP'10. Springer, Berlin, pp 301–312
- Melab N, Talbi EG, Cahon S, Alba E, Luque G (2006) Parallel metaheuristics: models and frameworks. In: Talbi EG (ed) Parallel combinatorial optimization. Wiley, New York, pp 149–161
- Micikevicius P (2004) General parallel computation on commodity graphics hardware: case study with the all-Pairs shortest paths problem. In: PDPTA, pp 1359–1365
- Mocholí J, Jaén J, Canós J (2005) A grid ant colony algorithm for the orienteering problem. In: The 2005 IEEE congress on evolutionary computation, vol 1, pp 942–949
- Munawar A, Wahib M, Munetomo M, Akama K (2009) Hybrid of genetic algorithm and local search to solve MAX-SAT problem using nVidia CUDA framework. *Genet Program Evolvable Mach*, pp 391–415
- O'Leary DP, Jung JH (2008) Implementing an interior point method for linear programs on a CPU-GPU system. *Electronic Transactions on Numerical Analysis* 28:174–189
- O'Neil M.A., Tamir D., Burtscher M. A Parallel GPU Version of the Traveling Salesman Problem. <http://www.gpucomputing.net/?q=node/12874>. Presentation at 'PDPTA'11 - The 2011 International Conference on Parallel and Distributed Processing Techniques and Applications'
- Pedemonte M, Nesmachnow S, Cancela H (2011) A survey on parallel ant colony optimization. *Applied Soft Computing* 11(8):5181–5197
- Ralphs TK, Ladányi L, Saltzman MJ (2003) Parallel Branch, Cut, and Price for Large-Scale Discrete Optimization. *Mathematical Programming* 98:253–280
- Ralphs T.K. (2006) Parallel Branch and Cut. In: E. Talbi (ed.) Parallel Combinatorial Optimization, pp. 53–101. Wiley, New York
- Robilliard D., Marion-Poty V., Fonlupt C. (2008) Population parallel GP on the G80 GPU. In: Proceedings of the 11th European conference on Genetic programming, EuroGP'08, pp. 98–109. Springer-Verlag, Berlin, Heidelberg
- Robilliard D., Marion V., Fonlupt C. (2009a) High performance genetic programming on GPU. In: Proceedings of the 2009 workshop on Bio-inspired algorithms for distributed systems, BADS '09, pp. 85–94. ACM, New York, NY, USA
- Robilliard D, Marion-Poty V, Fonlupt C (2009b) Genetic programming on graphics processing units. *Genet Program Evolvable Mach* 10(4):447–471
- Rocki K, Suda R (2012) Accelerating 2-opt and 3-opt local search using GPU in the travelling salesman problem. In: 2012 international conference on high performance computing and simulation (HPCS), pp 489–495

- Rostrup S, Srivastava S, Singhal K (2011) Fast and memory-efficient minimum spanning tree on the GPU. In: 2nd international workshop on GPUs and scientific applications (GPUScA 2011). Inderscience, Geneva
- Sanci S, Isler V (2011) A parallel algorithm for UAV flight route planning on GPU. *Int J Parallel Prog* 39(6):809–837
- Scheuermann B, So K, Guntsch M, Middendorf M, Diessel O, ElGindy H, Schmeck H (2004) FPGA implementation of population-based ant colony optimization. Special issue on hardware implementations of soft computing techniques. *Appl Soft Comput* 4(3):303–322
- Scheuermann B, Janson S, Middendorf M (2007) Hardware-oriented ant colony optimization. *J Syst Architect* 53(7):386–402
- Schulz C (2013) Efficient local search on the GPU—investigations on the vehicle routing problem. *J Parallel Distrib Comput* 73(1):14–31 (metaheuristics on GPUs)
- Solomon S, Thulasiraman P, Thulasiram R (2011) Collaborative multi-swarm PSO for task matching using graphics processing units. In: Proceedings of the 13th annual conference on genetic and evolutionary computation, GECCO '11. ACM, New York, pp 1563–1570
- Spampinato D, Elster A (2009) Linear optimization on modern GPUs. In: IEEE international symposium on parallel distributed processing, 2009. IPDPS 2009, pp 1–8
- Spiesens P, Manderick B (1991) A massively parallel genetic algorithm implementation and first analysis. In: Proceedings of 4th international conference on genetic algorithms
- Stivala A, Stuckey P, Wirth A (2010) Fast and accurate protein substructure searching with simulated annealing and GPUs. *BMC Bioinf* 11(1):446
- Talbi E (2006) Parallel combinatorial optimization. In: Wiley series on parallel and distributed computing. Wiley-Interscience, New York
- Tran QN (2010) Designing efficient many-core parallel algorithms for all-pairs shortest-paths using CUDA. In: Proceedings of the 2010 seventh international conference on information technology: new generations, ITNG '10. IEEE Computer Society, Washington, DC, pp 7–12
- Uchida A, Ito Y, Nakano K (2012) An efficient GPU implementation of ant colony optimization for the traveling salesman problem. In: Third international conference on networking and computing, pp 94–102
- Wang J, Dong J, Zhang C (2009) Implementation of ant colony algorithm based on GPU. In: Sixth international conference on computer graphics, imaging and visualization, 2009. CGIV '09, pp 50–53
- Weiss RM (2010) GPU-accelerated data mining with swarm intelligence. Honors thesis. Department of Computer Science, Macalester College. <http://metislogic.net/thesis.pdf>
- Wikipedia (2013) Streaming SIMD extensions. http://en.wikipedia.org/wiki/Streaming_SIMD_Extensions
- Wong ML, Wong TT, Fok KL (2005) Parallel evolutionary algorithms on graphics processing unit. In: The 2005 IEEE congress on evolutionary computation, 2005. vol 3, pp 2286–2293
- Wong ML, Wong TT (2006) Parallel hybrid genetic algorithms on consumer-level graphics hardware. In: IEEE congress on evolutionary computation, 2006. CEC 2006, pp 2973–2980
- Wong ML (2009) Parallel multi-objective evolutionary algorithms on graphics processing units. In: Proceedings of the 11th annual conference companion on genetic and evolutionary computation conference: late breaking papers, GECCO '09. ACM, New York, pp 2515–2522
- You YS (2009) Parallel ant system for traveling salesman problem on GPUs. <http://www.gpgpgpu.com/gecco2009>. Entry in 'GPUs for genetic and evolutionary computation' competition, GECCO 2009
- Yu Q, Chen C, Pan Z (2005) Parallel genetic algorithms on programmable graphics hardware. In: Wang L, Chen K, Ong Y (eds) ICNC 2005, LNCS, vol 3612, pp 1051–1059
- Zhao J, Liu Q, Wang W, Wei Z, Shi P (2011) A parallel immune algorithm for traveling salesman problem and its application on cold rolling scheduling. *Inf Sci* 181(7):1212–1223
- Zhu W, Curry J, Marquez A (2010) SIMD tabu search for the quadratic assignment problem with graphics hardware acceleration. *Int J Prod Res* 48(4):1035–1047